5. 가상 머신 실행 과정

개요

가상 머신(Virtual Machine, VM)은 컴파일러가 생성한 바이트코드를 실행하는 핵심 컴포넌트입니다. Monkey 언어의 VM은 스택 기반 아키텍처를 사용하여 효율적인 바이트코드 실행을 제공합니다. 이 문서에서는 VM의 구조, 스택 기반 실행 원리, 그리고 함수 호출 관리 시스템에 대해 상세히 설명합니다.

5.1 VM 구조체와 스택 기반 실행

VM 구조체 분석

VM 구조체는 바이트코드 실행에 필요한 모든 상태를 관리합니다:

```
type VM struct {
constants []object.Object // 상수 풀 - 컴파일 시 생성된 상수들
stack []object.Object // 연산 스택 - 계산 과정에서 사용
sp  int  // 스택 포인터 - 다음 값이 들어갈 위치
globals []object.Object // 전역 변수 저장소
frames []*Frame // 함수 호출 프레임 스택
framesIndex  int  // 현재 프레임 인덱스
}
```

각 필드의 역할

1. constants (상수 풀)

- 컴파일 시점에 생성된 모든 상수값들을 저장
- 정수, 문자열, 컴파일된 함수 등이 포함
- `OpConstant` 명령어로 스택에 로드됨

2. stack (연산 스택)

- 모든 연산의 중간 결과와 최종 결과를 저장
- 크기: `StackSize = 2048`
- LIFO(Last In, First Out) 방식으로 동작

3. sp (스택 포인터)

- 항상 다음 값이 들어갈 위치를 가리킴
- 스택의 최상단 요소는 `stack[sp-1]` 에 위치
- 0부터 시작하여 push 시 증가, pop 시 감소

4. globals (전역 변수)

- 전역 스코프의 변수들을 저장
- 크기: `GlobalSize = 65536`
- 인덱스를 통해 직접 접근 가능

5. frames (함수 호출 프레임)

- 각 함수 호출마다 생성되는 실행 컨텍스트
- 최대 `MaxFrames = 1024` 개까지 지원
- 재귀 호출과 중첩 함수 호출 관리

스택 기반 실행의 원리

스택 기반 아키텍처의 장점

1. 단순한 명령어 설계

```
// 예시: 1 + 2 계산
OpConstant 0 // stack: [1]
OpConstant 1 // stack: [1, 2]
OpAdd // stack: [3]
```

2. 메모리 효율성

- 임시 레지스터가 불필요
- 스택 하나로 모든 연산 처리
- 컴파일러 구현이 단순

3. 이식성

- 하드웨어 레지스터에 의존하지 않음
- 다양한 플랫폼에서 동일하게 동작

스택 관리 함수들

push() 함수

```go
func (vm *VM) push(o object.Object) error {
if vm.sp >= StackSize {
return fmt.Errorf("stack overflow")
}
vm.stack[vm.sp] = o
vm.sp++
return nil
```

```
}
```

pop() 함수

```
func (vm *VM) pop() object.Object {
o := vm.stack[vm.sp-1]
vm.sp--
return  o
}
```

VM 초기화 과정

```
func  New(bytecode *compiler.Bytecode) *VM {
mainFn := &object.CompiledFunction{Instructions: bytecode.Instructions}
mainClosure := &object.Closure{Fn: mainFn}
mainFrame := NewFrame(mainClosure, 0)

frames := make([]*Frame, MaxFrames)
frames[0] = mainFrame

return &VM{
constants: bytecode.Constants,
stack: make([]object.Object, StackSize),
sp: 0,
```

```
globals: make([]object.Object, GlobalSize),
frames: frames,
framesIndex: 1,
}
}
```

1. 메인 함수를 위한 `CompiledFunction` 생성
2. 클로저로 래핑하여 `mainClosure` 생성
3. 메인 프레임 생성 및 프레임 스택에 추가
4. 모든 필드 초기화

5.2 Run() 함수와 명령어 실행 루프

메인 실행 루프 구조

`Run()` 함수는 VM의 핵심으로, 바이트코드를 순차적으로 실행합니다:

```
func (vm *VM) Run() error {
var ip int
var ins code.Instructions
var op code.Opcode

for vm.currentFrame().ip < len(vm.currentFrame().Instructions())-1 {
vm.currentFrame().ip++
```

```
ip = vm.currentFrame().ip

ins = vm.currentFrame().Instructions()

op = code.Opcode(ins[ip])


switch op {

// 각 명령어별 처리 로직

}

}


return nil

}
```

실행 루프의 핵심 요소

1. 명령어 포인터(ip) 관리

- 현재 실행 중인 명령어의 위치
- 각 프레임마다 독립적으로 관리
- 점프 명령어로 변경 가능

2. 명령어 디코딩

- 바이트코드에서 Opcode 추출
- 필요한 경우 피연산자도 함께 읽기

3. 명령어별 분기 처리

- switch 문을 통한 효율적인 분기
- 각 명령어마다 특화된 처리 로직

주요 명령어별 실행 로직

상수 로드 (OpConstant)

```
case code.OpConstant:
constIndex := code.ReadUint16(ins[ip+1:])
vm.currentFrame().ip += 2

err := vm.push(vm.constants[constIndex])
if err != nil {
return err
}
```

산술 연산 (OpAdd, OpSub, OpMul, OpDiv)

```
case code.OpAdd, code.OpSub, code.OpMul, code.OpDiv:
err := vm.executeBinaryOperation(op)
if err != nil {
return err
}
```

조건부 점프 (OpJumpNotTruthy)

```
case code.OpJumpNotTruthy:
pos := int(code.ReadUint16(ins[ip+1:]))
```

```
vm.currentFrame().ip += 2


condition := vm.pop()
if !isTruthy(condition) {
vm.currentFrame().ip = pos - 1
}
```

전역 변수 접근 (OpGetGlobal, OpSetGlobal)

```
case  code.OpSetGlobal:
globalIndex := code.ReadUint16(ins[ip+1:])
vm.currentFrame().ip += 2
vm.globals[globalIndex] = vm.pop()


case  code.OpGetGlobal:
globalIndex := code.ReadUint16(ins[ip+1:])
vm.currentFrame().ip += 2
err := vm.push(vm.globals[globalIndex])
```

이진 연산 처리

```
func (vm *VM) executeBinaryOperation(op code.Opcode) error {
right := vm.pop()
```

```
left := vm.pop()


leftType := left.Type()

rightType := right.Type()


switch {

case  leftType == object.IntegerObj && rightType == object.IntegerObj:

return  vm.executeBinaryIntegerOperation(op, left, right)

case  leftType == object.StringObj && rightType == object.StringObj:

return  vm.executeBinaryStringOperation(op, left, right)

default:

return  fmt.Errorf("unsupported types for binary operation: %s %s",

leftType, rightType)

}

}
```

정수 연산 처리

```
func (vm *VM) executeBinaryIntegerOperation(op code.Opcode, left, right object.O

leftValue := left.(*object.Integer).Value

rightValue := right.(*object.Integer).Value


var  result  int64
```

```go
switch op {
case code.OpAdd:
result = leftValue + rightValue
case code.OpSub:
result = leftValue - rightValue
case code.OpMul:
result = leftValue * rightValue
case code.OpDiv:
result = leftValue / rightValue
}

return vm.push(&object.Integer{Value: result})
}
```

5.3 Frame 시스템과 함수 호출 관리

Frame 구조체 분석

```go
type Frame struct {
cl *object.Closure // 실행 중인 클로저
ip int // 명령어 포인터
basePointer int // 스택의 기준점
```

```
}
```

Frame의 역할

1. 함수별 실행 컨텍스트 분리

- 각 함수 호출마다 독립적인 실행 환경 제공
- 명령어 포인터와 지역 변수 영역 관리

2. 스택 프레임 관리

- `basePointer` 를 통해 지역 변수 영역 구분
- 함수 매개변수와 지역 변수의 스택 위치 계산

3. 클로저 지원

- 자유 변수(Free Variables) 접근 제공
- 렉시컬 스코프 구현

함수 호출 과정

1. 함수 호출 명령어 (OpCall)

```
case  code.OpCall:
numArgs := code.ReadUint8(ins[ip+1:])
vm.currentFrame().ip += 1


err := vm.executeCall(int(numArgs))
if  err != nil {
return  err
```

```
}
```

2. 호출 대상 결정

```
func (vm *VM) executeCall(numArgs int) error {
callee := vm.stack[vm.sp-1-numArgs]
switch callee := callee.(type) {
case *object.Closure:
return vm.callClosure(callee, numArgs)
case *object.Builtin:
return vm.callBuiltin(callee, numArgs)
default:
return fmt.Errorf("calling non-function and non-built-in")
}
}
```

3. 클로저 호출 처리

```
func (vm *VM) callClosure(cl *object.Closure, numArgs int) error {
if numArgs != cl.Fn.NumParameters {
return fmt.Errorf("wrong number of arguments: want=%d, got=%d",
cl.Fn.NumParameters, numArgs)
}
```

```
frame := NewFrame(cl, vm.sp-numArgs)
vm.pushFrame(frame)

vm.sp = frame.basePointer + cl.Fn.NumLocals

return  nil
}
```

지역 변수와 매개변수 처리

지역 변수 설정 (OpSetLocal)

```
case  code.OpSetLocal:
localIndex := code.ReadUint8(ins[ip+1:])
vm.currentFrame().ip += 1

frame := vm.currentFrame()
vm.stack[frame.basePointer+int(localIndex)] = vm.pop()
```

지역 변수 읽기 (OpGetLocal)

```
case  code.OpGetLocal:
localIndex := code.ReadUint8(ins[ip+1:])
vm.currentFrame().ip += 1
```

```
frame := vm.currentFrame()

err := vm.push(vm.stack[frame.basePointer+int(localIndex)])
```

함수 반환 처리

값과 함께 반환 (OpReturnValue)

```
case  code.OpReturnValue:

returnValue := vm.pop()


frame := vm.popFrame()

vm.sp = frame.basePointer - 1


err := vm.push(returnValue)

if  err != nil {

return  err

}
```

값 없이 반환 (OpReturn)

```
case  code.OpReturn:

frame := vm.popFrame()

vm.sp = frame.basePointer - 1
```

```
err := vm.push(Null)
if err != nil {
return err
}
```

프레임 관리 함수들

현재 프레임 접근

```
func (vm *VM) currentFrame() *Frame {
return vm.frames[vm.framesIndex-1]
}
```

프레임 추가

```
func (vm *VM) pushFrame(f *Frame) {
vm.frames[vm.framesIndex] = f
vm.framesIndex++
}
```

프레임 제거

```
func (vm *VM) popFrame() *Frame {
vm.framesIndex--
return vm.frames[vm.framesIndex]
```

```
}
```

클로저와 자유 변수

클로저 생성 (OpClosure)

```
case  code.OpClosure:
constIndex := code.ReadUint16(ins[ip+1:])
numFree := code.ReadUint8(ins[ip+3:])
vm.currentFrame().ip += 3

err := vm.pushClosure(int(constIndex), int(numFree))
```

자유 변수 접근 (OpGetFree)

```
case  code.OpGetFree:
freeIndex := code.ReadUint8(ins[ip+1:])
vm.currentFrame().ip += 1

currentClosure := vm.currentFrame().cl
err := vm.push(currentClosure.Free[freeIndex])
```

실행 예시: 함수 호출 과정

다음 Monkey 코드의 실행 과정을 살펴보겠습니다:

```
let add = fn(a, b) { a + b };

add(1, 2);
```

1. 함수 정의 단계

1. `OpClosure` - add 함수 클로저 생성
2. `OpSetGlobal` - 전역 변수 add에 저장

2. 함수 호출 단계

1. `OpGetGlobal` - add 함수 로드
2. `OpConstant` - 인수 1 로드
3. `OpConstant` - 인수 2 로드
4. `OpCall` - 함수 호출 (인수 2개)

3. 함수 실행 단계

1. 새 프레임 생성 (basePointer 설정)
2. 매개변수 a, b가 스택의 적절한 위치에 배치
3. 함수 본문 실행: `OpGetLocal a`, `OpGetLocal b`, `OpAdd`
4. `OpReturnValue` - 결과 반환

4. 반환 후 정리

1. 프레임 제거
2. 스택 포인터 복원
3. 반환값을 스택에 푸시

정리

VM의 스택 기반 실행 시스템은 다음과 같은 특징을 가집니다:

1. **단순하고 효율적인 명령어 처리**: 각 명령어가 스택을 통해 일관된 방식으로 동작
2. **체계적인 함수 호출 관리**: Frame 시스템을 통한 독립적인 실행 컨텍스트 제공
3. **메모리 효율성**: 스택 기반 아키텍처로 최소한의 메모리 사용
4. **확장 가능성**: 새로운 명령어 추가가 용이한 구조

이러한 설계를 통해 Monkey 언어는 효율적이고 안정적인 바이트코드 실행 환경을 제공합니다.